Topic : Stack operations using an Array (C programming)

Data structure Used.

The program implements a stack using an Array in C. A stack follows the LIFO (Last in, First out) principle, which means the last element inserted is the first element removed.

Variables and Initial Values.

Variable	Initial value	purpose.
Size	5	Maximum capacity of the stack.
Stack (Size)	All elements intialized to 0	stores stack elements.
top	-1	indicates an empty stack
item	uninitialized	stores the returned value from Pop().
i	uninitialized	controls the display loop.

Functions Used.

1) Push (item): Inserts an element into the stack if Space is available.
2) Pop (): Removes and returns the top element if the Stack is not empty.
3) display (): Prints all elements currently in the stack.
4) main (): Calls the functions in the specified order.

Conditions and Loop:

- If (top == Size -1) checks for stack overflow.
- if (top == -1) checks for stack underflow or an empty stack.
- for (i = 0; i <= top; i++) prints each elements from the bottom to the top.

Manual Trace Table.

Step	operation	Item/Return Value	top	stack content
1.	Initial state	-	-1	Empty
2.	push(10)	10	0	10
3.	push(20)	20	1	10, 20
4.	push(30)	30	2	10, 20, 30
5)	display()	-	2	10, 20, 30
6.	pop()	30	1	10, 20
7.	Push(40)	40	2	10, 20, 40
8.	Push(50)	50	3	10, 20, 40, 50
9.	Push(60)	60	4	10, 20, 40, 50, 60
10.	Push(70)	70(attempted)	4	10, 20, 40 50, 60.
11.	display()	-	4	10, 20, 40, 50, 60.
12.	Pop()	60	3	10, 20, 40, 50.
13.	Pop()	50	2	10, 20, 40.
14.	Pop()	40	1	10, 20.
15.	Pop()	20	0	10.
16.	Pop()	10	-1	Empty.
17.	display()	-	-1	Empty.
18.	pop()	-1	-1	Empty..

The push(70) operation fails because the stack is full. The final pop() fails because the stack is empty..

Loop Variable Tracing.

The display() function Contains this loop.

```c
for (i=0; i <= top; i ++).
```

First display.

At this point, top = 2.

i	condition	Element printed
0	True	10
1	True	20
2	True	30
3	False	None

Output : Stack : 10 20 30.

Second display.

At this point top = 4.

i	condition	Element printed.
0	True	10
1	True	20
2	True	40
3	True	50
4)	True	60
5	False	None.

Output : Stack : 10 20 40 50 60.

Third display.

Here top = -1

The stack is empty, so the function prints stacks is Empty.

The loop does not execute.

Predicted output.

Pushed : 10
pushed : 20
Pushed : 30
Stack : 10 20 30
Popped : 30
Pushed : 40
Pushed : 50
Pushed : 60.
Stack overflow
Stack : 10 20 40 50 60
Popped : 60
Popped : 50
Popped : 40.

Popped : 20
Popped : 10.
Stack is Empty.
Stack underflow
Popped : -1.

Final stack state:

- Final value of top = -1
- Final logical stack contents = Empty.
- Stack capacity = 5
- overflow occurrences = 1
- underflow occurrences = 1

What - if Analysis

Original Statement

```
</> C
Push (70):
```

Modified Statement

```
</> C
push (25):
```

Trace.

Property	original	Modified.
operation	Push(70)	push (25)
top befor operation	4	4
Stack before operation	10 20 40 50 60	10,20 40 50 60.
Result	overflow	overflow.
top after operation	4	4.

Error Analysis

The pop() function returns -1 when the stack is empty.

```
</> C
if (top == -1)
{
    printf ("Stack underflow\n");
    return -1;
}
```

However, -1 could also be a valid element in Stored in the stack. This makes it difficult for the calling function to distinguish an error from a Successfully popped value of -1.

</> C

```
int pop (int * item)
{  if (top == = -1)
       return 0;
   * item = Stack [top];
   top - -;
    return 1;
}
```

The caller Should check the returned status before printing the popped value.

Three Additional Test cases.

Test case 1: Push and pop one element

</> C

```
push (10);
item = pop();
printf (" popped : %d /n ", item);
display ();
```

output

```
pushed : 10
popped  : 10
Stack is Empty
```

Test case 2 : Push five elements.

</> C

```
push(1);
push(2);
push (3);
push (4);
push (5);
display();
```

Expected output.

Pushed : 1
Pushed : 2
Pushed : 3
Pushed : 4
Pushed : 5
Stack : 1 2 3 4 5.

Test case 3: Pop from an empty stack.

```
item = pop();
printf("popped : %d\n", item);
```

Expected output:

Stack underflow.
Popped : -1

After executing each test, record the actual output and indicate whether it matches your prediction.

Conclusion:

The program demonstrates Stack operations using an array inc. It follows the LIFO principle, where the last inserted elements is removed first. The Push(), pop() and display() functions perform insertion, deletion, and display operations.

The program correctly detects stack overflow when the stack is full and stack underflow when the stack is empty. Manual tracing helps us understand changes in the top variable; stack contents and function return values before executing the program.

The final Stack is empty, and the final value of top is -1. The predicted output can be verified by executing the program using a C compiler.